

CAC Controller

Technische Dokumentation — Pioneer CAC CD-Automatenwechsler Web Controller

Inhalt

- 1. Hardware und Anschluss
- 2. Pioneer CAC Serielles Protokoll
- 3. Software-Architektur
- 4. Frontend
- 5. Datenfluss
- 6. Datenbank-Details
- 7. JSON-Import
- 8. Deployment
- 9. Sicherheit
- 10. Bekannte Einschränkungen

1. Hardware und Anschluss

1.1 Unterstützte Modelle

Modell	Kapazität	Player	Baudrate	Lautstärke	Geschwindigkeit
CAC-V3000	300 CDs	2	9600	Ja	Ja
CAC-V3200	300 CDs	2	9600	Ja	Ja
CAC-V5000	500 CDs	2	9600	Ja	Ja
CAC-V180M	18 CDs	1	4800	Nein	Nein

1.2 Serielle Verbindung

Parameter	Wert
Baudrate	9600 (V3000/V3200/V5000) oder 4800 (V180M)
Datenbits	8
Stoppbits	1
Paritaet	Keine
Terminator	CR (\r , ASCII 0x0D)
Max. Befehlslaenge	20 Zeichen

Physikalischer Anschluss: 15-poliger D-Sub-Stecker (RS-232C) am Wechsler, verbunden ueber einen USB-zu-Seriell-Adapter mit dem Raspberry Pi.

1.3 Systemarchitektur

```

Browser (Smartphone / Tablet / PC)
|
| HTTP (REST API) + WebSocket
v
Node.js Server (Raspberry Pi)
|
| RS-232C Serial (9600 Baud, 8N1)
v
Pioneer CAC CD-Automatenwechsler

```

2. Pioneer CAC Serielles Protokoll

2.1 Befehlsformat

Alle Befehle werden im Format `<Player-ID>PS<Befehl>\r` gesendet.

Beispiel: `1PS001ZSPL\r` — waehlt Player 1, laedt Disc 001 und startet die Wiedergabe.

2.2 Ausfuehrungsbefehle (27)

Befehl	Code	Beschreibung	Argumente
Player Select	PS	Player auswaehlen	1 oder 2
Disc Select	ZS	CD laden	001-500

Disc Return	ZR	CD zuruecklegen	—
Start	SA	Motor starten	—
Reject	RJ	Motor stoppen	—
Play	PL	Wiedergabe	—
Pause	PA	Pause	—
Scan Forward	NF	Vorwaerts scannen	—
Scan Reverse	NR	Rueckwaerts scannen	—
Search	SE	Suche ausfuehren	—
Track	TR	Track waehlen	01-99
Volume	VL	Lautstaerke	0-255
Speed	SP	Geschwindigkeit	90-110%
Fade	FD	Lautstaerke-Fade	0-255 Zielwert
Duration	DU	Fade-Dauer	1-99 Sekunden
Stop Marker	SM	Stopppunkt	—
Changer Reset	!!	Wechsler Reset	—

2.3 Abfragebefehle (14)

Befehl	Code	Antwort	Beschreibung
Job Status	?J	R / B	Bereit / Beschaeftigt
Player Mode	?P	P01 - P22	Aktueller Modus
Disc Number	?Z	3-stellig / XXX	Geladene CD
Track Number	?R	2-stellig / XX	Aktueller Track
Time Code	?T	4-stellig (MMSS)	Verstrichene Zeit
TOC Info	?Q	10-stellig	Tracks + Lead-Out
Model Name	?X	6 Zeichen	Modellbezeichnung
Mech Error	?E	E00 - E99	Fehlercodes
Disc Status	?K	8 Flags	Disc-Status

2.4 Player-Modi

Code	Modus	Beschreibung
------	-------	--------------

P01	Park	CD geladen, Motor aus
P02	Set Up	TOC wird gelesen
P03	Reject	Motor wird gestoppt
P04	Play	Wiedergabe laeuft
P06	Pause	Pausiert
P07	Search	Suchvorgang
P08	Scan	Scan-Vorgang
P20	Disc Unset	Keine CD geladen
P21	Load	CD wird geladen
P22	Unload	CD wird zurueckgelegt

2.5 Lautstaerke-Mapping

dB	Wert	dB	Wert
0 dB	255	-30 dB	138
-1 dB	247	-40 dB	97
-3 dB	239	-50 dB	60
-6 dB	230	-60 dB	16
-10 dB	213	-70 dB	5
-20 dB	174	Stumm	0

3. Software-Architektur

3.1 Technologie-Stack

Komponente	Technologie	Version
Runtime	Node.js	≥ 18.x
Web-Framework	Express	5.x
WebSocket	ws	8.x
Datenbank	better-sqlite3	11.x
Serielle Komm.	serialport	12.x

Frontend	Vanilla JS (SPA)	—
i18n	Eigene Loesung	DE/EN

3.2 Projektstruktur

```

cac-controller/
├─ server.js           # Einstiegspunkt
├─ package.json
├─ cac-controller.service # Systemd-Unit
├─ src/
│   ├─ protocol.js     # Pioneer-Protokoll
│   ├─ serial.js        # Serielle Verbindung
│   ├─ player.js        # Player-Manager
│   ├─ scanner.js       # CD-Scanner
│   ├─ musicbrainz.js   # MusicBrainz-API
│   ├─ database.js      # SQLite-Datenbank
│   ├─ routes.js        # REST-API
│   └─ websocket.js     # WebSocket-Manager
├─ public/
│   ├─ index.html       # SPA
│   ├─ css/app.css      # Styling
│   ├─ js/app.js        # Frontend-Logik
│   ├─ js/i18n.js       # Uebersetzungen
│   └─ covers/          # Cover-Bilder
└─ data/
    └─ cac-controller.db # Datenbank

```

3.3 Module im Detail

server.js — Einstiegspunkt

Initialisiert alle Komponenten: Datenbank → Serielle Verbindung → PlayerManager → Scanner → Express → WebSocket. Graceful Shutdown ueber SIGINT/SIGTERM.

protocol.js — Pioneer-Protokoll

Enthaelt die komplette Protokolldefinition: Modell-Eigenschaften, Befehls-Konstanten, Response-Parser. Der Parser erkennt automatisch Fehlercodes, Modi, Track-/Disc-Nummern, Zeitcodes, TOC-Info und mehr.

serial.js — Serielle Verbindung

Verwaltet die RS-232C-Verbindung mit automatischer Wiederverbindung (alle 5 Sekunden), separaten Befehlsqueues pro Player (200ms Abstand) und Priority-Queue fuer zeitkritische

Befehle. Fire-and-Forget-Architektur: Befehle und Antworten sind komplett entkoppelt.

player.js — Player-Manager

Zentrales Modul mit ~800 Zeilen. Verwaltet den Zustand beider Player, pollt den Wechsler in versetzten Intervallen und implementiert vier Wiedergabe-Modi:

- Automatischer Wechsel zum anderen Player bei Disc-Ende
- Pre-Cueing des anderen Players fuer nahtlose Uebergaenge
- CD, beide Player oder gesamter Wechsler
- Server-seitige Engine mit intelligentem Dual-Player-Management und CD-Pre-Loading

scanner.js — CD-Scanner

Scannt Slots: CD laden → Warten → Motor starten → TOC abfragen → Daten speichern → CD zuruecklegen. Echtzeit-Fortschritt per EventEmitter/WebSocket. Pausiert das Polling waehrend des Scans.

musicbrainz.js — MusicBrainz-Integration

Rate-limitierte API-Aufrufe (1,1s Abstand). Suche per Freitext, Kuenstler+Titel, Barcode, Track-Anzahl+Dauer oder Disc-ID. Cover-Art ueber das Cover Art Archive. Automatisches Mapping von MusicBrainz-Daten in das interne Format.

database.js — SQLite-Datenbank

WAL-Modus fuer Performance. UPSERT mit COALESCE (nur uebergebene Felder werden aktualisiert). Kaskadierendes Loeschen fuer Tracks. Standard-Einstellungen beim ersten Start.

routes.js — REST-API

50+ Endpunkte fuer Player-Steuerung, Library-CRUD, Cover-Upload, MusicBrainz-Suche, Scanner, Playlists, Favoriten, Bewertungen, History, Suche, Statistiken, Einstellungen und JSON-Import.

websocket.js — WebSocket-Manager

Broadcast aller Zustandsänderungen an alle verbundenen Clients. Sendet initialen Zustand bei Verbindung.

4. Frontend

4.1 Single Page Application

Das Frontend ist eine SPA ohne Build-Tools oder Frameworks. Drei Dateien: `index.html` (Struktur), `app.js` (Logik), `i18n.js` (Uebersetzungen), `app.css` (Dark Theme).

4.2 Internationalisierung

Drei Ebenen: HTML-Attribute (`data-i18n`), JavaScript (`t('key')`), Server-Nachrichten (strukturierte Objekte). ~200 Uebersetzungsschlüssel fuer Deutsch und Englisch.

4.3 Echtzeit-Updates

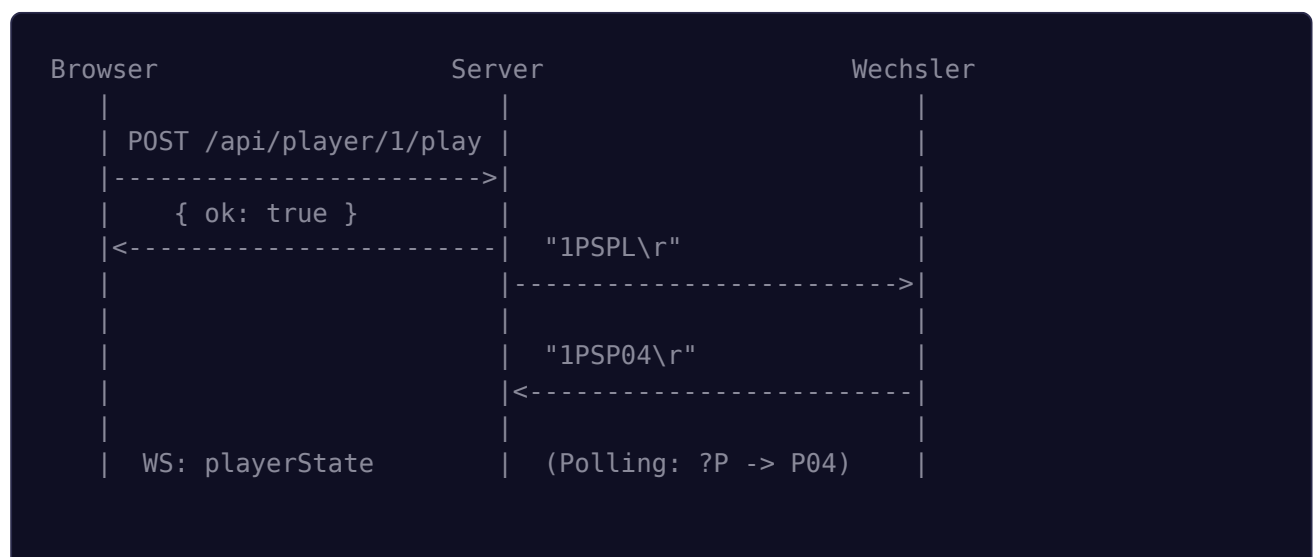
WebSocket liefert sofortige Updates fuer Player-Status, Scanner-Fortschritt und Playlist-Aenderungen. Die Zeitanzeige wird clientseitig interpoliert fuer fluessige Darstellung.

4.4 Cover-Upload

Clientseitige Verarbeitung: Dateiauswahl/Drag&Drop → Canvas-Resize (max 500x500) → JPEG 85% → Base64-Upload → Server speichert in `public/covers/`.

5. Datenfluss

5.1 Player-Steuerung



```
| { mode: 'P04' } |
|<=====|
```

5.2 CD-Scan

Browser	Server	Wechsler
POST /api/scanner/scan		
{ slot: 42 }		
----->	"1PS042ZS\r"	
WS: scanProgress		
{ status: 'loading' }	(wartet auf P01/P02...)	
<=====		
	"1PSSA\r"	
WS: scanProgress		
{ status: 'reading' }		
<=====	"1PS?Q\r"	
	"1PS0112045000\r"	
	<-----	
WS: scanProgress	(TOC: 12 Tracks, 45:00)	
{ totalTracks: 12 }		
<=====	"1PSRJ\r" + "1PSZR\r"	
	----->	

5.3 Playlist mit Dual-Player

Playlist: [Slot 42/Track 3, Slot 101/Track 1, Slot 42/Track 7]

Schritt 1: Lade CD 42 in Player 1, spiele Track 3
 Gleichzeitig: Lade CD 101 in Player 2 (Pre-Load)

Schritt 2: CD 101 bereits in Player 2
 Stoppe Player 1, spiele Track 1 auf Player 2
 Player 1 hat noch CD 42 (kein Pre-Load noetig)

Schritt 3: CD 42 noch in Player 1
 Stoppe Player 2, spiele Track 7 auf Player 1

6. Datenbank-Details

6.1 Schema


```

cds (
  slot INTEGER PRIMARY KEY,      -- 1-500
  disc_id TEXT,                  -- MusicBrainz Disc-ID
  title TEXT, artist TEXT,      -- Album-Metadaten
  year TEXT, genre TEXT,
  total_tracks INTEGER,
  total_duration_seconds INTEGER,
  cover_url TEXT,               -- Pfad zum Cover
  musicbrainz_release_id TEXT,
  barcode TEXT, label TEXT, country TEXT,
  created_at TEXT, updated_at TEXT
)

tracks (
  slot INTEGER REFERENCES cds(slot) ON DELETE CASCADE,
  track_number INTEGER,
  title TEXT, artist TEXT,
  duration_seconds INTEGER, isrc TEXT,
  UNIQUE(slot, track_number)
)

playlists, playlist_items, play_history,
favorites, ratings, settings

```

6.2 WAL-Modus

Write-Ahead Logging fuer bessere Performance bei gleichzeitigem Lesen und Schreiben.

6.3 UPSERT mit COALESCE

Nur uebergebene Felder werden aktualisiert — bestehende Werte bleiben bei NULL erhalten.

7. JSON-Import

7.1 Unterstuetzte Formate

Array von CDs, Objekt mit `cds`-Array oder Objekt mit Slot-Nummern als Keys.

Automatische Feld-Zuordnung fuer verschiedene Bezeichnungen (`slot` / `cd_number` / `nr` , `title` / `album` / `name` , etc.).

7.2 Vorschau-UI

Editierbare Tabelle mit Status-Badges (Neu/Update/Warnung), aenderbaren Slot-Nummern (Auto-Swap bei Duplikaten), Einzelauswahl per Checkbox und Mojibake-Reparatur.

8. Deployment

8.1 Systemd-Service

```
[Service]
Type=simple
User=pi
WorkingDirectory=/opt/cac-controller
ExecStart=/usr/bin/node server.js
Restart=always
RestartSec=5
SupplementaryGroups=dialout
Environment=NODE_ENV=production
```

8.2 Installation

```
git clone https://github.com/NFDiJee/cac-controller.git /opt/cac-controller
cd /opt/cac-controller
npm install
sudo cp cac-controller.service /etc/systemd/system/
sudo systemctl enable --now cac-controller
```

9. Sicherheit

- Keine Authentifizierung (nur fuer LAN gedacht)
- Kein HTTPS (unverschluesst)
- Empfehlung: Firewall, kein Port-Forwarding, bei Bedarf Nginx mit Basic Auth

10. Bekannte Einschränkungen

1. Der Pioneer liefert nur Gesamtdauer und Track-Anzahl. Individuelle Track-Laengen nur ueber MusicBrainz.

2. CD-Laden dauert 10-30 Sekunden. Vollstaendiger Scan $\approx$ mehrere Stunden.
 3. Aktuell wird ein Wechsler gesteuert. Multi-Changer (Hub-Architektur) ist geplant.
 4. Audioausgang ist analog am Geraet. Kein digitales Streaming.
-

CAC Controller v1.0.0 — Copyright © 2025 Dirk Jensen

Basierend auf dem Pioneer CAC-V3000 Programming Manual (Version 2.0, Mai 1993)

MIT License